



DEPARTMENT OF OCEAN OPERATIONS AND CIVIL ENGINEERING

AIS4104 ROBOTICS WITH PROJECT

Autonomous Exploring Rover

Authors:

Khadija Rauf

William Haugland

Youssef Abdul Ghafour

May 2026

Table of Contents

Preface	i
List of Figures	iv
List of Tables	v
1 Introduction	1
2 Theory	1
2.1 SLAM	1
2.2 Autonomous Exploration	2
2.3 CV	5
2.3.1 Lens distortions	5
2.3.2 Pose Estimation	5
2.3.3 Perspective-n-Point	5
2.4 Tranformation Matrixis	5
2.5 Path Planning	5
2.5.1 Path Planning Problem	5
2.5.2 Environment Representation	5
2.5.3 Graph and Connectivity	6
2.5.4 Cost Modeling	6
2.5.5 Dijkstra Algorithm	6
2.5.6 A* Algorithm	7
2.5.7 Heuristic Function	7
2.6 Navigation and Integration	8
3 Method and Material	9
3.1 Method	9
3.2 Tools and Materials	9
3.3 SLAM	9
3.4 Autonomous Exploration Implementation	10
3.5 CV	11

3.5.1	Camera Calibration	12
3.5.2	ArUco Detection	13
3.5.3	Mapping and Localization	14
3.5.4	Testing and Consolidation	14
3.5.5		15
3.6	Path Planning	15
3.6.1	Initial approach: Dijkstra planner	15
3.6.2	Grid-based A* (4-connectivity)	16
3.6.3	Extension to 8-connectivity	17
3.6.4	Local wall penalty	17
3.6.5	Obstacle inflation	18
3.6.6	Path simplification	18
3.6.7	Distance field and centering cost	18
3.6.8	Use of AI-generated code for distance field	19
3.7	Navigation and Integration	20
4	Results	21
4.1	SLAM and Autonomous Exploration	22
4.2	CV	24
4.3	Path planning results	24
4.3.1	Dijkstra planner	25
4.3.2	A* with 4-connectivity	25
4.3.3	A* with 8-connectivity	27
4.3.4	A* with local wall penalty	29
4.3.5	A* with obstacle inflation	29
4.3.6	A* with distance field	30
4.4	Navigation and Integration	32
5	Discussion	32
5.1	SLAM	32
5.2	CV	32
5.3	Path Planning	32

5.3.1	Dijkstra planner	32
5.3.2	A* with 4-connectivity	33
5.3.3	A* with 8-connectivity	33
5.3.4	A* with local wall penalty	34
5.3.5	A* with obstacle inflation	34
5.3.6	A* with distance field	35
5.4	Navigation and Integration	36
6	Conclusion	36
	Bibliography	37
	Appendices	i

List of Figures

2.1	Relationship between mapping, localization, motion control, and integrated robotic approaches, adapted from [1, p. 2].	1
2.2	Graphical representation by Bayesian network of full-SLAM [1, p. 55]. . . .	2
2.3	Agents interact with environments through sensors and actuators [4, pp. 35].	3
2.4	Model-based utility-based agent architecture, [4, p. 54].	3
3.1	Comparison image of the provided and working calibration papers	12
3.2	Finished camera calibration	13
3.3	Manually constructed waypoint graph on the SLAM map. Nodes represent selected waypoints and edges represent valid connections used by the Dijkstra planner.	16
4.1	Evaluation of mapping performance under manual control and the <code>auto_explorer</code> node within a cluttered workspace containing dense obstacles.	22
4.2	Comparison of the SLAM-generated occupancy grid (a) against the official MazeMap architectural layout (b) for a sparse, low-clutter environment. . .	23
4.3	Corridor mapping using the autonomous exploration (<code>auto_explorer</code>) node.	24
4.4	Terminal output of the Dijkstra planner showing the computed path from node 0 to node 4.	25
4.5	Paths generated using the 4-connectivity A* planner for three different start and goal configurations in RViz.	26
4.6	Paths generated using the 8-connectivity A* planner for three different start and goal configurations in RViz (same as 4-connectivity).	28
4.7	Path generated using the A* planner from node 0 to node 4 with local wall penalty in RViz.	29
4.8	Paths generated using the A* planner from node 0 to node 4 with obstacle inflation for different inflation radii.	30
4.9	Paths generated using the A* planner with the distance field approach for different start and goal configurations in RViz.	31
4.10	Comparison of the total number of nodes in the generated paths from node 0 to node 4 for the previous planner and the final distance field-based planner.	31
4.11	Enter Caption	32
5.1	Zoomed view of the path near the start position showing the effect of corridor width variation.	35
5.2	Zoomed view of the path at a corridor intersection showing the shift in the center of free space.	36

List of Tables

4.1	Comparison of the number of nodes for different test configurations.	32
-----	--	----

1 Introduction

This is where the intro is

- Goal - Motivation - What was done

2 Theory

This where the theory is.

2.1 SLAM

Simultaneous Localization and Mapping (SLAM) is the problem of estimating a robot's pose while simultaneously constructing a map of an unknown environment. The problem is central to autonomous mobile robotics because localization and mapping are mutually dependent: accurate mapping requires a reliable pose estimate, while localization requires a consistent representation of the environment [1].

From a system perspective, SLAM belongs to a broader integrated robotic approach connecting mapping, localization, exploration, and motion control [1]. This is important because a mobile robot cannot solve autonomous mapping only through a mapping algorithm; it must also decide where to move, avoid obstacles, and maintain a usable estimate of its own pose.

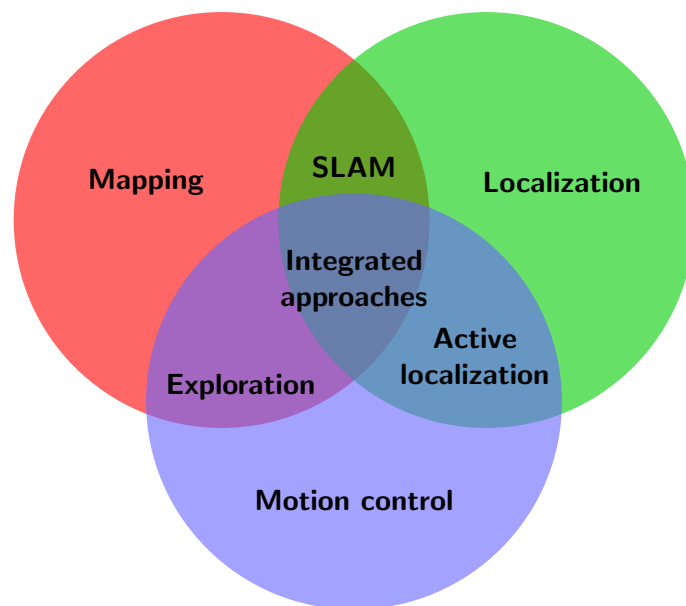


Figure 2.1: Relationship between mapping, localization, motion control, and integrated robotic approaches, adapted from [1, p. 2].

SLAM is commonly formulated as a probabilistic estimation problem. The objective is to estimate the joint posterior distribution over the robot pose and the map:

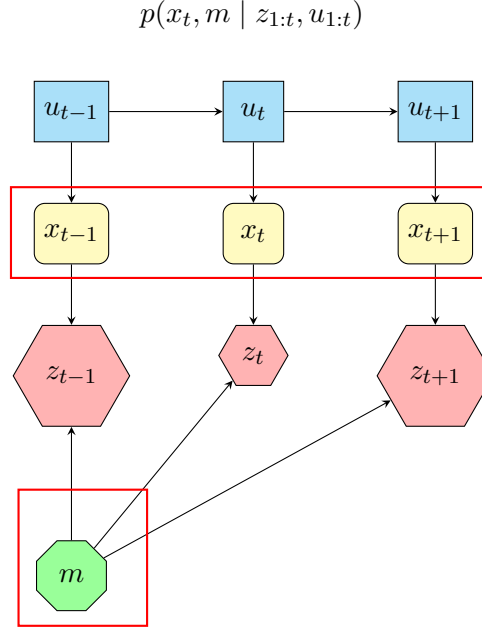


Figure 2.2: Graphical representation by Bayesian network of full-SLAM [1, p. 55].

where x_t is the robot pose at time t , m is the map, $z_{1:t}$ are sensor observations, and $u_{1:t}$ are control inputs. This formulation follows the probabilistic robotics framework, where the motion model predicts the state transition and the observation model corrects the estimate using sensor measurements [1, p. 40].

Several SLAM paradigms exist, including Kalman filter-based SLAM, particle filter-based SLAM, and graph-based SLAM [1]. In this project, the relevant category is particle filter-based SLAM, since the ROS `slam_gmapping` package is based on Rao–Blackwellized particle filtering. This approach separates the estimation problem by sampling possible robot trajectories while maintaining a map estimate conditioned on each trajectory.

A common map representation for mobile robot SLAM is the occupancy grid. In an occupancy grid, the environment is discretized into cells, where each cell represents whether the corresponding region is free, occupied, or unknown. This representation is suitable for laser-based 2D mapping and provides a direct interface to later navigation and exploration algorithms [1].

2.2 Autonomous Exploration

Autonomous exploration addresses the problem of deciding where the robot should move while the map is still incomplete. In the context of SLAM, exploration is not separate from mapping; it is the behaviour that drives the robot toward informative parts of the environment so that the SLAM process receives new sensor observations.

This can be interpreted using the intelligent-agent framework. An agent perceives the environment through sensors and acts upon it through actuators [4].

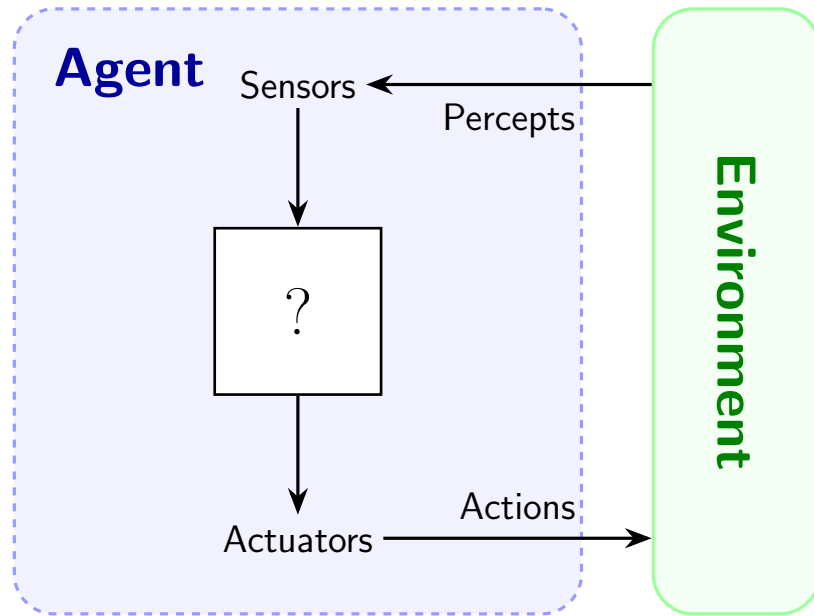


Figure 2.3: Agents interact with environments through sensors and actuators [4, pp. 35].

For the robot in this project, the percepts include laser scans, odometry, the occupancy grid map, and the transform between the map and robot frame. The actions are velocity commands sent to the robot through `/cmd_vel`.

The environment is partially observable, dynamic, sequential, and initially unknown. The robot cannot observe the full map at once, each movement affects future observations, and new obstacles or narrow passages may appear during operation. According to Russell and Norvig, such environments require agents to maintain internal state and use models of how the world changes and how actions affect the world [4].

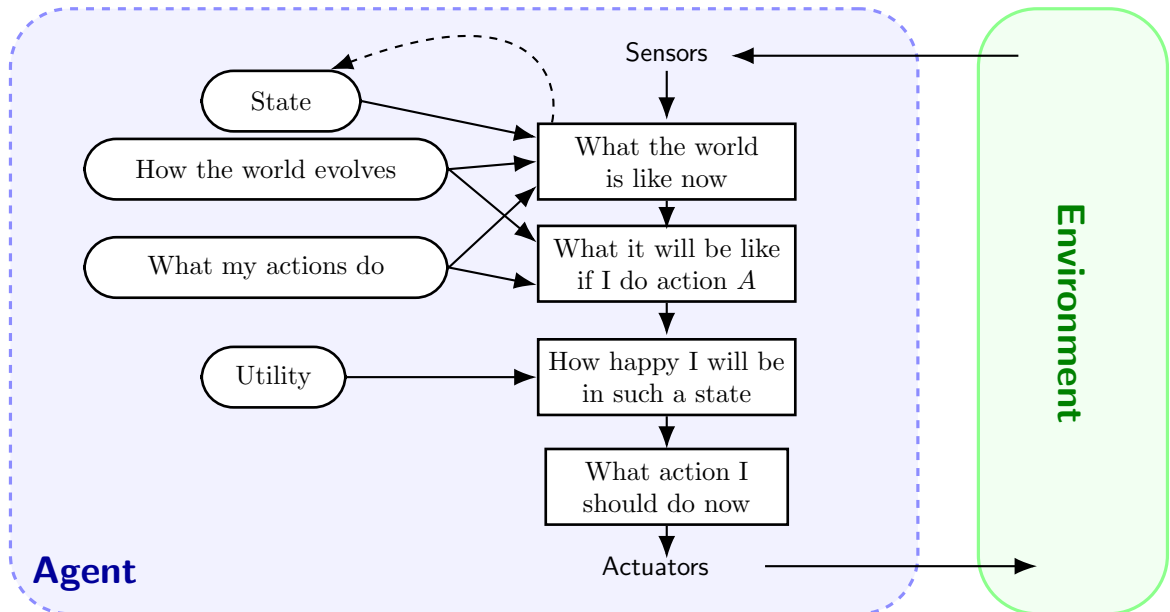


Figure 2.4: Model-based utility-based agent architecture, [4, p. 54].

The exploration controller can therefore be described as a model-based, goal-directed,

utility-inspired agent. It is model-based because it maintains internal information such as the current map, robot pose, laser-sector distances, visited areas, and blacklisted failed goals. It is goal-directed because it selects frontier goals that represent useful unexplored regions. It is utility-inspired because candidate frontiers and short-horizon trajectories are ranked using heuristic scoring functions rather than selected randomly.

A common approach for autonomous mapping is frontier-based exploration. A frontier is the boundary between known free space and unknown space. Moving toward frontiers is useful because it tends to increase map coverage while keeping the robot inside already traversable space [8]. In this project, a frontier cell is defined as:

$$\text{frontier}(m_x, m_y) = \text{free}(m_x, m_y) \wedge \text{hasUnknownNeighbor}(m_x, m_y)$$

where (m_x, m_y) denotes a cell in the occupancy grid. This definition is not a complete exploration algorithm by itself; it only identifies candidate cells. The final behaviour depends on clustering, scoring, local motion selection, and recovery logic.

The use of heuristic evaluation is consistent with search-based AI principles, where heuristic functions are used to estimate the desirability of candidate states or actions when exhaustive search is computationally infeasible [4, p. 92–102].

In classical search algorithms such as A*, candidate nodes are evaluated using:

$$f(n) = g(n) + h(n)$$

where $g(n)$ represents the cost to reach node n , and $h(n)$ is a heuristic estimate of the remaining cost to the goal. This formulation provides a principled way to guide search toward promising regions of the state space.

In contrast, the exploration problem addressed in this project operates in an unknown and dynamically evolving environment, where a complete graph representation is not available. Therefore, instead of performing optimal graph search, the system uses a heuristic scoring approach to evaluate candidate frontier regions and motion trajectories.

The scoring functions used in this project can be interpreted as a generalized evaluation function, where multiple criteria are combined to approximate utility. These include distance to the goal, heading alignment, obstacle clearance, information gain (through unknown-space density), and penalties for repeated visits.

From an AI perspective, this corresponds to an approximate utility-based decision mechanism. This interpretation is consistent with the concept of expected utility in decision-making under uncertainty:

$$EU(a) = \sum_s P(s \mid a) U(s)$$

where $EU(a)$ is the expected utility of action a , $P(s \mid a)$ represents the probability of reaching state s after taking action a , and $U(s)$ is the utility assigned to that state [4, p. 611].

In the context of this project, the exact probability distribution over future states is not explicitly computed. Instead, the heuristic scoring functions act as an approximation of

this expected utility, combining multiple criteria such as distance, safety, and exploration benefit into a single evaluation value.

2.3 CV

2.3.1 Lens distortions

Cameras use lenses to focus the light that they capture. When light passes through the lens it gets distorted from its linear path and a noticeable radial distortion happens[5].

2.3.2 Pose Estimation

2.3.3 Perspective-n-Point

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R|t] \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

2.4 Transformation Matrix

Rodrigues' formula `cv.Rodrigues(rvec)`

2.5 Path Planning

2.5.1 Path Planning Problem

Path planning is a fundamental problem in robotics. The goal is to find a path that allows a robot to move from a starting position to a target position while avoiding obstacles. The path should be collision-free and, in most cases, optimal with respect to a given cost function.

From a theoretical point of view, this problem can be seen as a search problem in a configuration space, where all possible positions of the robot are considered [2]. In practical applications, this space is often simplified into a discrete representation.

A common approach is to represent the environment as a grid, where each cell corresponds to a small area of space and is classified as either free or occupied. This allows the path planning problem to be solved using graph-based search algorithms.

2.5.2 Environment Representation

To apply path planning algorithms, the environment must be represented in a form that can be processed by a computer. A common approach is to discretize the space into a finite number of elements, reducing the complexity of the problem.

A widely used representation is the occupancy grid, where the environment is divided into cells of equal size. Each cell contains information about whether the corresponding area is free or occupied by an obstacle. This representation is particularly suitable for environments obtained through SLAM systems.

Occupancy grids are commonly used in robotics because they provide a simple and efficient way to represent uncertain environments and enable the use of graph-based planning methods [3].

2.5.3 Graph and Connectivity

Once the environment is discretized, it can be modeled as a graph. In this representation, each cell corresponds to a node, and edges represent possible transitions between neighboring cells.

Formally, the graph can be defined as $G = (V, E)$, where V is the set of nodes and E is the set of edges. Path planning then consists of finding a sequence of nodes that connects the start to the goal while minimizing a cost function [2].

The notion of connectivity defines which neighboring cells are reachable. In grid-based environments, two common types are used. In 4-connectivity, each cell is connected to its horizontal and vertical neighbors. In 8-connectivity, diagonal neighbors are also included.

From a theoretical perspective, increasing connectivity allows the discrete graph to better approximate the continuous space, leading to paths that are closer to straight-line solutions [2, 3].

2.5.4 Cost Modeling

In graph-based path planning, each transition between nodes is associated with a cost. The objective is to find a path that minimizes the total accumulated cost.

The total cost of a path is defined as:

$$C = \sum_{i=1}^N c_i$$

where c_i is the cost of each step, and N is the number of steps.

In grid-based environments, the cost typically reflects the distance between cells. Horizontal and vertical movements have a lower cost, while diagonal movements have a higher cost due to their longer length.

More advanced models also include additional costs related to the environment, such as proximity to obstacles. This allows the planner to account not only for path length but also for safety, which is an important consideration in robotics applications [2].

2.5.5 Dijkstra Algorithm

The Dijkstra algorithm is a classical shortest-path algorithm used in graph theory. It computes the minimum cost path from a start node to all other nodes in the graph.

The algorithm explores nodes in order of increasing cost, ensuring that once a node is selected, the shortest path to that node has been found. This guarantees optimality when all edge costs are non-negative.

However, Dijkstra does not use any information about the goal location. As a result, it may explore a large number of nodes, which makes it less efficient in large environments.

Despite this limitation, it is widely used as a reference method due to its simplicity and theoretical guarantees [2].

2.5.6 A* Algorithm

The A* algorithm is a widely used method for path planning in graph-based environments. It is designed to find a path with the lowest cost from a start node to a goal node. Compared to Dijkstra's algorithm, A* improves efficiency by using additional information to guide the search toward the goal.

At each step, A* evaluates nodes using the following function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost from the start node to the current node n , and $h(n)$ is a heuristic function that estimates the cost from n to the goal. The value $f(n)$ represents the total estimated cost of a path going through node n .

The heuristic function plays an important role, as it guides the search toward the goal. From a theoretical point of view, if the heuristic is admissible, meaning that it never overestimates the true cost to the goal, the A* algorithm is guaranteed to find an optimal path [2].

The algorithm maintains a set of nodes to be explored and always selects the node with the lowest $f(n)$ value. By combining the actual cost and an estimate of the remaining distance, A* is able to guide the search efficiently toward the goal while reducing the number of explored nodes compared to Dijkstra's algorithm [2, 3].

2.5.7 Heuristic Function

The heuristic function plays a key role in the performance of the A* algorithm. It provides an estimate of the distance from the current node to the goal.

In grid-based environments, a common choice is the Manhattan distance:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

This heuristic is admissible when movement is restricted to horizontal and vertical directions, ensuring that A* remains optimal.

Choosing an appropriate heuristic is important, as it directly affects the efficiency of the search [2].

2.6 Navigation and Integration

Autonomous navigation requires more than computing a path between two points. A mobile robot must estimate its pose in a known map, plan a feasible route to a target, and execute motion commands while continuously avoiding obstacles. Navigation therefore combines localization, planning, perception, coordinate transformations, and motion control into one integrated system.

In a grid-based navigation system, the map produced by SLAM can be reused as the reference environment for localization and path planning. The occupancy grid provides information about free, occupied, and unknown space, while the robot pose defines the starting point for the planner. A path planner can then compute a sequence of poses from the current robot position to the goal position using graph-search principles, as described in the path planning theory section.

Localization is required because the planner assumes that the robot pose is known in the map frame. In practical mobile robot navigation, localization is often treated probabilistically because odometry and sensor measurements contain uncertainty. This is consistent with the probabilistic robotics view of estimating the robot state from motion and sensor observations [1]. In this project, the relevant localization concept is particle-filter localization, where multiple pose hypotheses are maintained and updated using sensor data.

Visual landmarks can support localization and navigation by providing identifiable reference points in the environment. A fiducial marker such as an ArUco marker can be detected by a camera and used to estimate the relative pose between the marker and the camera. If the marker position is already known in the map, the observed marker pose can be used to infer the robot pose through coordinate transformations. This connects computer vision with robotic localization.

Coordinate transformations are therefore essential in an integrated navigation system. The robot must maintain consistent relationships between frames such as `map`, `odom`, `base_footprint`, and the camera frame. In matrix form, transformations allow poses measured in one coordinate frame to be expressed in another. This is necessary when combining camera-based marker observations with map-based navigation and robot motion.

From a systems perspective, navigation integration means that different modules must exchange information through a common representation. The map provides the spatial reference, localization estimates the robot state, the planner computes the route, the local controller generates velocity commands, and perception modules provide semantic or corrective information. Without this integration, each subsystem may work independently, but the robot will not be able to perform a complete autonomous task.

In ROS-based navigation, this integration is commonly organized through a navigation stack, where a global planner computes a path over the map, a local planner converts this path into feasible motion commands, and costmaps represent static and dynamic obstacles around the robot. This layered structure separates long-range route planning from short-range obstacle avoidance and control.

3 Method and Material

3.1 Method

This is where the method and material part is

3.2 Tools and Materials

Add the Material part here as well (hardware), from Pre Project report

chat did everything! material list from pre report. Did we get anything else that's not in the pre report

starting phase -report -choosing project -setting goals - pre project report -Jettracer tutorials (setup the cars) - Locking IP addresses - Getting an other car - Reflashing the other car

3.3 SLAM

The mapping system was implemented using ROS and the `slam_gmapping` package, utilizing laser scan data from `/scan` and odometry from `/odom` to construct a 2D occupancy grid map published on `/map`. To oversee this process, a custom ROS node, `youssef_slam`, was developed to monitor map generation and automate the finalization of the session. Rather than replacing the underlying SLAM algorithm, this supervisor node was designed to evaluate the map's stability and determine the optimal moment to conclude the mapping task.

The node subscribes to the occupancy grid map and compares consecutive map updates. Let M_t and M_{t-1} denote two consecutive occupancy grid maps. A custom map-change metric was defined as:

$$C = \sum_{i=1}^N \mathbb{I}(M_t(i) \neq M_{t-1}(i))$$

where N is the number of grid cells and $\mathbb{I}(\cdot)$ is an indicator function that returns 1 if the cell value changed and 0 otherwise.

This metric was developed specifically for this project as a practical engineering heuristic and is not part of standard SLAM formulations. The underlying assumption is that as the mapping process converges, successive occupancy grid updates become increasingly similar. If the number of changed cells remains below a predefined threshold for a specified duration, and a minimum exploration time has elapsed, the map is considered stable.

After stability is detected, the node performs the following sequence:

1. publishes an exploration-complete signal on `/exploration_done`;
2. stops the robot by publishing zero velocity commands;
3. saves the map using `map_server`;

-
4. copies the generated map files to the target robot directory;
 5. stops the `slam_gmapping` node.

This design automated map finalization by removing the need for manual map saving and manual termination of the SLAM process.

3.4 Autonomous Exploration Implementation

The autonomous exploration behaviour was implemented in the custom ROS node `auto_explorer`. This node was designed as an online control loop that continuously reads sensor and map information, selects an exploration target, generates local motion commands, and applies safety recovery behaviours.

The node subscribes to:

- `/scan` for laser-based obstacle perception;
- `/odom` for odometry-based progress estimation;
- `/map` for occupancy-grid exploration;
- `/exploration_done` to stop when the SLAM monitor has completed the mapping session.

It also uses the transform between the map frame and the robot base frame to estimate the robot pose in the map. The output of the node is a velocity command published to `/cmd_vel`.

The exploration behaviour was structured into four main stages.

First, frontier cells are extracted from the occupancy grid using the condition:

$$\text{frontier}(m_x, m_y) = \text{free}(m_x, m_y) \wedge \text{hasUnknownNeighbor}(m_x, m_y)$$

This identifies cells that are safe to stand near while being adjacent to unknown space.

Second, connected frontier cells are grouped into clusters using queue-based breadth-first search. Very small clusters are rejected to avoid unstable or noisy goals.

Third, each frontier cluster is scored using a project-specific heuristic function:

$$S = w_s \log(1 + n) - w_d d + w_h h + w_u u + w_b b - w_v v - w_n p_n$$

where n is the number of cells in the frontier cluster, d is the distance from the robot to the candidate goal, h is the heading alignment term, u is the unknown-density score, b is a same-goal bonus, v is the visit-memory penalty, and p_n is the narrow-area penalty.

This scoring function defines a multi-criteria evaluation of candidate exploration goals, combining geometric, informational, and behavioural factors. The weights were selected

empirically based on system performance during real-world testing, allowing the robot to balance exploration efficiency, safety, and robustness.

Unlike classical graph-based planners such as A*, which assume a fully known environment, this formulation operates directly on a partially known and dynamically evolving map. As a result, the scoring function prioritizes informative and reachable frontiers rather than computing globally optimal paths.

Fourth, after a frontier goal is selected, the node computes a local waypoint and evaluates short-horizon velocity candidates. Each candidate trajectory is forward-simulated and rejected if it violates occupancy or safety constraints. The remaining candidates are scored using:

$$J = w_g g + w_p p + w_c c + w_s s + b_c - w_v v_r - p_r$$

where g is final heading alignment to the waypoint, p is progress toward the waypoint, c is laser clearance, s is smoothness, b_c is a corridor-centering bonus, v_r is the visit penalty, and p_r is a reverse-motion penalty.

This short-horizon evaluation forms a local decision layer that connects high-level goal selection with low-level motion execution. By combining forward simulation with heuristic scoring, the system generates safe and effective velocity commands in real time without requiring a full optimal control formulation.

In addition to goal and trajectory scoring, the node includes safety and recovery logic. Laser scans are divided into sectors such as front, side, diagonal, and rear regions. If the front sector becomes blocked, or if odometry indicates that the robot has not made sufficient progress, the controller enters a recovery state. Depending on rear clearance, corridor detection, and side clearance, the robot either turns, reverses, or performs an escape turn.

The final behaviour can therefore be summarized as a model-based heuristic exploration agent. It uses SLAM output as its world model, frontier detection as its goal-generation mechanism, heuristic scoring as its decision mechanism, and laser-sector safety gates as its reactive layer.

3.5 CV

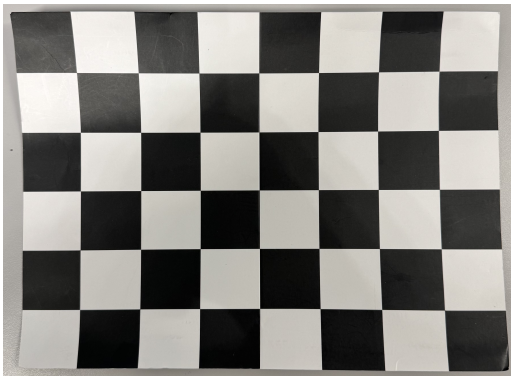
Camera vision with ROS encompass multiple different problems for implementation. This resulted in multiple iterations and gradual development over time. An extensive library from OpenCV, handles all off the camera processing and visualization to extract ArUco pose estimation for further processing. The first version of the project used four different scripts to show the camera feed, process ArUco detection and map their location, save the map location to a yaml file and localize the car from the yaml file waypoints. This meant that multiple scripts was doing camera processing at ones, leading to crashes and nodes shutting downs because of CPU overload. The final project uses a single script for camera, mapping and localisation with a helper script to save the waypoint to a waypoints.yaml file.

3.5.1 Camera Calibration

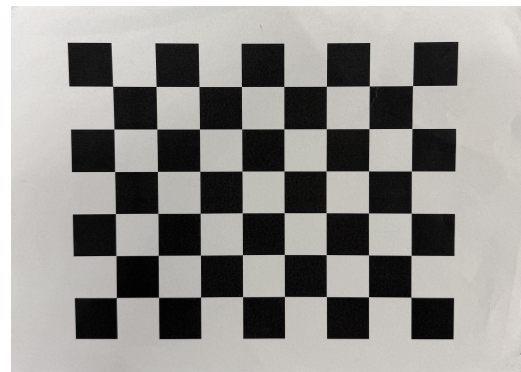
To ensure accurate pose estimation with ArUco codes the first step was to ensure that the camera was correctly calibrated. Following the Waveshare Wiki JetRacer ROS Kit [6], Tutorial VIII: Start the Camera Node [7] to enable and make sure the camera feed is publishing to Rviz. Then the camera calibration was activated using:

```
1 rosrn camera_calibration cameracalibrator.py --size 5x7 --square  
0.03 image:=/csi_cam_0/image_raw camera:=/csi_cam_0
```

When trying to calibrate the camera with the provided calibration paper from the Jetracer box it didn't give a good score and scanning seemed random when holding the paper in front of the camera. Even at different angles and distances the squares where not detected. Trying to tape the paper flat to the wall didn't fix the detection issues but the bars where filled enough to save and calibrate the camera. After the camera was calibrated the camera node and Rviz was launched to inspect the cameras corners for lens distortions. It reviled that the calibration performed was not good enough with distortions in the corners of the image.



(3.1a) Calibration paper without borders



(3.1b) Calibration paper with borders

Figure 3.1: Comparison image of the provided and working calibration papers

The problem with provided calibration paper shown in Figure 3.1 is that the calibration requires white borders around the outer set of squares to correctly identify the paper. The paper also needs to lay flat to the wall, but the bulges bulges in the thick plastic made this difficult. Taping the calibration paper to a white wall and holding the Jetracer at different positions and angles didn't seem to help either and the next few calibration attempts where worse then the first one. The solution was to print out a new calibration paper and tape it flat to the wall.

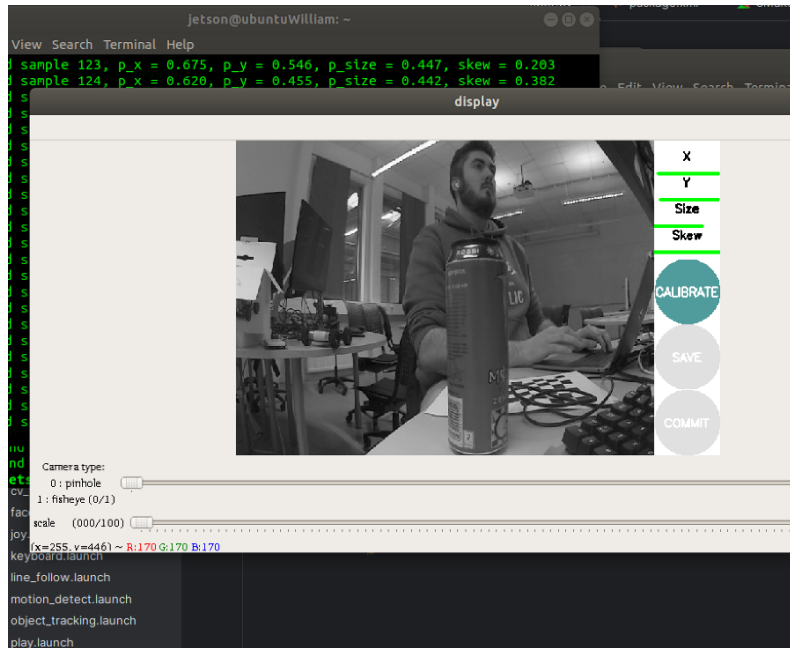


Figure 3.2: Finished camera calibration

This improved the calibration sampling. By moving the camera horizontally to the extremes of right and left, the sampling of the X variable was improved. The by moving the camera from the extremes of up and down the Y variable improved. Moving the camera further away and closer to the paper improved the Size variable. At a certain distance the calibration paper didn't get detected. Skew was improved by rotating the camera 360° directly in front of the paper and rotating the camera to different angles in different x and y positions. The finished camera calibration is shown in Figure 3.2 with max score for X, Y and skew and a smaller but green score non the less. The "Calibrate" but was then pressed. After calibration was finished with its 145 samples, it was saved with the "SAVE" button and the "COMMIT" button pushes the camera calibration to the Jetracer. The camera node was then inspected in Rviz and showed that the camera calibration was successful with no lens distortion.

3.5.2 ArUco Detection

The Jetracer ROS AI KIT has pre written scripts in *src/jetracer_ros/scripts*. The *cv_video.py* was analysed and used as inspiration when subscribing to the camera node, converting to greyscale and showing viewing window. These scripts utilizes rospy, which makes python able to use the ROS middleware features. Then OpenCV cv2, cv2.aruco, cv_bridge and numpy as imported to utilize camera vision with ArUco code detection.

The first step to achieve ArUco detection was to subscribe to the camera node using:

```
1 self.info_sub = rospy.Subscriber(self.camera_name+ "/camera_info",
    CameraInfo, self.info_callback)
```

Then a sheet of ArUco marker was acquired and measured. Each square measured 38cm each side and was initialized using the *rospy.get_param* shown below:

```
1 self.marker_size = rospy.get_param( ~marker_size , 0.038)
```

The raw image feed from the subscribed camera node is then converted to greyscale which is required for OpenCV ArUco detection. This is done for better processing performance and the ArUco detection only rely on finding sharp black and white corners. After converting to grayscale the ArUco detection is performed with OpenCV's `aruco` "`aruco.detectMarkers`", which checks for corners and id. If an ID is detected, "`aruco.estimatePoseSingleMarkers`" is activated which uses the corners, `marker_size`, `camera_matrix` and `dist_coeffs`, which is the distortion coefficients of how the lens bends light. Then the values for `tvec` was printed to the terminal.

For the visualization of the pose, the raw image feed is compressed and drawn using OpenCV and `bgr8`. `Bgr8` enables it to draw borders around the markers and visualize the pose with three axis's.

```
1 aruco.drawDetectedMarkers(img, corners, ids)
2 for i in range(len(ids)):
3     marker_id = int(ids[i][0])
4     aruco.drawAxis(img, self.camera_matrix, self.dist_coeffs, rvecs[i],
        tvecs[i], 0.05)
```

The terminal prints out ArUco ID with coordinates in x right, y down and z forward. The "`aruco.estimatePoseSingleMarkers`" post coordinates and rotation in "`tvec`", a 1x3 matrix (x,y,z), and `rvec` a 1x3 matrix (rx,ry,rz). "`rvec`" is not in Euler angles and "`tvec`" uses a different coordinate/rotation system then ROS. ROS uses euler angles for rotation and x forward, y left and z up for its coordinates. To convert the coordinates from OpenCV to ROS coordinates and rotation, the project first converts the 1x3 `tvec` matrix in to single variables (x,y,z), `x_dist, y_dist, z_dist = tvec[0], tvec[1], tvec[2]`. The the ROS x is equal to OpenCV z, y is equal to -x and z is equal to -y, as this gives the required coordinate system to use further use with ROS.

3.5.3 Mapping and Localization

The next step was to implement waypoint saving for the markers. A new script was created to save the markers to a `waypoint.yaml` file for further implementation with path planning and navigation. The idea is to subscribe to the messages sent from `aruco_mapper.py` and save detected ArUco code positions to this `waypoints.yaml` file as `PointStamped`. It then subscribed to this `PointStamped` node and collected the x,y coordinate of the detected marker with its id. If a new id was detected it would update the `waypoints.yaml` file or skip it if the id was already detected.

A new script was written to import the `waypoints.yaml` file to use for pose estimation and localization of the Jetracer. This worked by processing the camera feed, detecting ArUco codes and calculating the pose. The calculated pose was then published to `/initialpose` to update the Jetracers position in real-time if it had drifted from its estimated Odom position.

3.5.4 Testing and Consolidation

Testing the scripts by launching singular or two files worked great and the envisioned functionalities worked as intended. A launch file was created to enable multiple scripts

to be launched at ones for testing purposes. A problem was occurring while testing the scripts together. They nodes shut downs unexpectedly shortly after launching and the Rviz node would crash shortly after. The problem was probably that three of the five scripts launched did the same camera processing and publishing three compressed camera feeds with identical names. Debugging took some time, but it was concluded that the CPU of the Jetracer overloaded and nodes would crash because of the overloaded CPU. When designing the scripts they where mostly meant to work independently or with a single other script.

The crashes showed a design issue with the code structure. Multiple scripts doing the same camera processing, ArUco detection, publishing compresses camera feed and ArUco drawing for visual testing as well as the base *cv_video.py*, renamed *ris_cv_video.py*, publishing and opening a camera window. A consolidation of the necessary scripts was the best solution to ensure no duplicate processes and *ris_cv_video.py* was dropped as the new script would use Rviz instead of launching a separate view window.

3.5.5

3.6 Path Planning

The path planning component was developed step by step, starting from a simple approach and gradually improving the planner to achieve better path quality and safer trajectories.

3.6.1 Initial approach: Dijkstra planner

The first path planning approach was based on Dijkstra’s algorithm. This method was chosen as an initial step because it is simple and well-known, making it suitable for understanding the basic workflow of path planning before moving to a more advanced grid-based planner.

The map was first generated using an existing SLAM package and visualized in RViz. Waypoints were then manually selected by clicking points along the corridor. These waypoints were used to create a graph, where each waypoint represents a node and each valid connection between two waypoints represents an edge.

The waypoint graph was stored in a YAML file and loaded into ROS parameters. Each waypoint was defined by an ID and its map coordinates, while each edge was defined by the two connected waypoint IDs. In the code, the edge cost was computed using the Euclidean distance between the coordinates of the connected waypoints.

After loading the data, the graph was converted into an undirected adjacency list representation. Dijkstra’s algorithm was then applied using a priority queue, where the node with the lowest accumulated cost was expanded first. When the goal node was reached, the final path was reconstructed by following the stored parent nodes from the goal back to the start.

The constructed waypoint graph is shown in Figure 3.3.

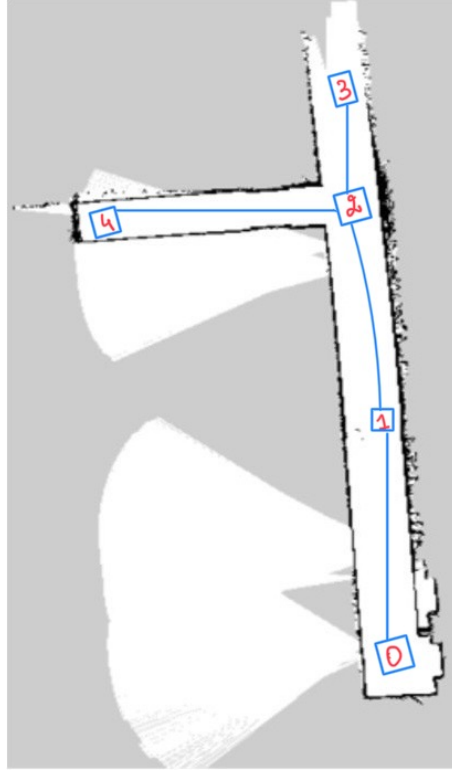


Figure 3.3: Manually constructed waypoint graph on the SLAM map. Nodes represent selected waypoints and edges represent valid connections used by the Dijkstra planner.

This approach produced valid paths and was useful for understanding graph-based planning and how to work with map data. However, it required manual placement of waypoints and connections, which limited its flexibility in more complex environments.

The implementation of the graph-based planner can be found in the project repository under `Ais4103-Jetracer-ROS-AI-KIT/src/ris/src/graph_planner`.

3.6.2 Grid-based A* (4-connectivity)

To improve flexibility and avoid manual graph construction, the planner was redesigned to operate directly on the occupancy grid produced by the SLAM system. In this representation, each grid cell corresponds to a node, and the planner searches for a path between a start and a goal position.

The A* algorithm was used for path planning, combining the cost from the start node and a heuristic estimate to the goal. A Manhattan distance heuristic was used, which is well suited for grid-based environments with axis-aligned motion.

The initial implementation used a 4-connectivity model, where movement is only allowed in the horizontal and vertical directions. This means that from a given cell, the robot can only move to its four direct neighbors (up, down, left, and right).

A simple binary cost model was applied. Free cells were assigned a constant traversal cost, while occupied cells were treated as obstacles and excluded from the search. As a result, the planner focused only on finding a valid path without considering distance to obstacles.

The planner was implemented by maintaining an open list of nodes to be explored and a closed list of visited nodes. At each step, the node with the lowest total cost was selected, where the total cost is the sum of the path cost from the start and the heuristic estimate to the goal. The cost of moving from one cell to a neighboring cell was constant in this configuration.

The path was reconstructed by storing parent nodes during the search process, allowing the final trajectory to be obtained once the goal was reached.

This implementation was developed in `Ais4103-Jetracer-ROS-AI-KIT/src/ris/src/grid_planner/plan_` and served as a baseline for the subsequent improvements described in the following sections.

3.6.3 Extension to 8-connectivity

The next step was to extend the planner from 4-connectivity to 8-connectivity. In the previous version, each cell could only be connected to its four direct neighbors. With 8-connectivity, diagonal neighbors are also included, so each cell can have up to eight possible neighboring cells.

This was implemented by defining relative offsets for the neighboring cells. For each current cell, the planner checks the possible neighbors by applying horizontal, vertical, and diagonal displacements. This only changes the neighbor expansion part of the A* algorithm, while the general search structure remains the same.

A corner-cutting check was also added before accepting diagonal moves. When a diagonal neighbor is evaluated, the two adjacent horizontal and vertical cells are checked first. If one of these cells is occupied, the diagonal move is rejected.

Different movement costs were used depending on the direction of movement. Horizontal and vertical moves were given a cost of 10, while diagonal moves were given a cost of 14. The value 10 was used as a scaled cost for moving one grid cell in a straight direction. A diagonal move corresponds to moving approximately $\sqrt{2}$ times farther than a straight move. Since $10 \cdot \sqrt{2} \approx 14.1$, the diagonal cost was approximated as 14.

This allowed the planner to account for the longer distance of diagonal movements while keeping all movement costs as integers.

3.6.4 Local wall penalty

To reduce the tendency of the planner to generate paths close to obstacles, an additional cost was added based on how close a cell is to nearby walls.

For each candidate cell, a small neighborhood around it was checked to see if there were obstacles nearby. This was implemented in the function `int wallPenalty(int gx, int gy)`, which scans the surrounding cells based on the grid coordinates.

If obstacles were found close to the cell, an extra cost was added. The more obstacles present in the neighborhood, the higher the penalty. As a result, cells located near walls become more expensive to traverse than cells in open space.

This modification changes the cost used in the A* algorithm, so that the planner not only

looks for short paths, but also takes into account the proximity to obstacles.

3.6.5 Obstacle inflation

To increase the safety margin around obstacles, obstacle inflation was added to the planner. The idea was to avoid not only the occupied cells, but also the free cells that are too close to walls.

This was implemented in the function `bool isCellFreeInflated(int gx, int gy)`. Instead of checking only the candidate cell, the function checks a square neighborhood around it using an inflation radius. If an occupied cell is found inside this neighborhood, the candidate cell is treated as blocked and is not used by the planner.

The inflation radius was defined in grid cells and tested with different values. In practice, a radius of 10 cells was selected, as it provided better clearance from obstacles compared to smaller values.

This check was used inside the A* algorithm when evaluating neighboring cells. As a result, the planner only considers cells that are free after inflation, instead of only checking whether the original cell is occupied.

3.6.6 Path simplification

After the A* search, a path simplification step was added to reduce unnecessary intermediate points in the generated path.

This was implemented in the function `simplifyPath()`. The function analyzes the direction between consecutive points in the path by comparing the relative movement between grid cells. If several points lie on the same straight line, the intermediate points are removed, while points where the direction changes are kept.

As a result, the final path contains fewer waypoints while preserving the overall shape of the trajectory. This makes the path easier to visualize and more suitable for execution by the robot.

3.6.7 Distance field and centering cost

In the final version of the planner, a distance field was introduced to improve how obstacle proximity is handled during path planning.

The distance field stores, for each cell in the grid, the distance to the nearest obstacle. This is computed once before running the A* algorithm. In this implementation, obstacle cells are first initialized with a distance of zero, and then the distance values are propagated to neighboring cells using a breadth-first search (BFS). As a result, each cell contains the minimum distance to the closest obstacle.

This approach was implemented in `Ais4103-Jetracer-ROS-AI-KIT/src/ris/src/grid_planner/plan_tes` based on code generated by the Claude AI tool. The generated code was adapted to integrate with the existing occupancy grid and ROS structure.

During the A* search, this precomputed distance is used directly when evaluating each

candidate cell. Instead of scanning a neighborhood as in the previous implementation, the planner retrieves the distance value and computes a penalty based on it. This was implemented through an updated version of the function `wallPenalty(int gx, int gy)`, where the cost depends on the distance to the nearest obstacle rather than counting nearby occupied cells.

The distance field is also used to handle obstacle inflation more efficiently. The function `isCellFreeInflated(int gx, int gy)` checks whether the distance of a cell is above a chosen threshold. If the distance is too small, the cell is considered too close to an obstacle and is rejected.

This results in a more continuous cost model, where all cells are evaluated based on their distance to obstacles. Compared to the previous approach, this avoids repeated neighborhood scans and allows the planner to evaluate each cell more efficiently during the search.

3.6.8 Use of AI-generated code for distance field

The implementation of the distance field was based on code generated using the Claude AI tool. The generated code was adapted and integrated into the existing planner so that it works correctly with the ROS framework and the occupancy grid used in this project.

The main idea of the distance field is to assign, to each cell in the grid, a value that represents how far it is from the nearest obstacle. This information is computed once before running the A* algorithm and then reused during the search. In the previous implementation, the distance to obstacles was estimated locally by scanning neighboring cells every time a new cell was evaluated. This resulted in repeated computations and made the cost evaluation more complex.

To compute the distance field, the algorithm first identifies all obstacle cells in the occupancy grid and assigns them a distance value of zero. All other cells are initialized with a large value. Then, a breadth-first search (BFS) is performed starting from all obstacle cells at the same time. A queue is used to process the cells, and at each step, the algorithm takes one cell from the queue and updates its neighboring cells. If a smaller distance to an obstacle is found for a neighbor, the value is updated and the neighbor is added to the queue. This process continues until all cells in the grid have been assigned their minimum distance to the closest obstacle.

Once the distance field is computed, it can be used directly during the A* search. When evaluating a candidate cell, the planner simply retrieves the precomputed distance value instead of scanning its neighborhood. This makes the computation faster and also simplifies the structure of the code, since the same information is reused everywhere.

The function `wallPenalty(int gx, int gy)` was modified to use this distance information. In the previous version, this function checked a neighborhood around the cell and counted how many obstacles were present to compute a penalty. In the new version, it directly uses the distance value of the cell and converts it into a cost. Cells that are closer to obstacles receive a higher penalty, while cells that are farther away receive a lower cost. This results in a smoother and more continuous cost model compared to the previous approach.

The function `isCellFreeInflated(int gx, int gy)` was also updated. Previously, this function checked whether any obstacle was present in a local neighborhood around the

cell. In the new version, it simply compares the distance value to a predefined threshold. If the distance is smaller than this threshold, the cell is considered too close to an obstacle and is rejected. Otherwise, it is considered valid. This replaces the previous inflation method with a simpler and more efficient check.

Overall, this approach makes use of a more global representation of the environment. Instead of relying only on local checks during the search, the planner has access to distance information for all cells. This not only simplifies the implementation, but also allows the same data to be reused for both cost computation and obstacle avoidance.

3.7 Navigation and Integration

The complete system was divided into two operational phases: a mapping phase and a navigation phase. This separation was necessary because the first phase generated the spatial data required by the second phase. The mapping phase produced both the occupancy grid map and the ArUco waypoint database, while the navigation phase reused these files for localization and goal-directed navigation.

In the first phase, the system was launched using `slam_and_vision.launch`. This launch file started `slam_gmapping`, the custom `youssef_slam` supervisor node, the `auto_explorer` node, the ArUco processor, and the waypoint saver. During this phase, `slam_gmapping` generated the occupancy grid map from laser scan and odometry data, while `auto_explorer` moved the robot through the environment. At the same time, `aruco_processor.py` ran in mapping mode, where detected ArUco markers were transformed into the map frame and published as waypoints. These waypoints were then stored by `waypoint_saver.py` in `aruco_waypoints.yaml`.

The output of the first phase was therefore a reusable navigation dataset consisting of the saved map files, `mymap.pgm` and `mymap.yaml`, together with the semantic marker waypoint file `aruco_waypoints.yaml`. This meant that the map did not only describe obstacles and free space, but also contained stored semantic target locations identified by marker IDs.

In the second phase, the system was launched using the navigation bringup launch file. This phase loaded the saved map using `map_server`, started AMCL for localization, and launched `move_base` for path planning and motion execution. The ArUco processor was switched from mapping mode to localization mode. In this mode, marker 1 was used as a bootstrap landmark: after repeated valid detections within the configured distance range, the robot pose was calculated from the known marker pose and published to `/initialpose`. This allowed AMCL to initialize the robot pose in the map frame.

The custom A* planner was integrated into `move_base` as a `nav_core::BaseGlobalPlanner` plugin. Earlier standalone planner versions were used to test world-to-grid conversion, grid-to-world conversion, 4-connectivity, 8-connectivity, obstacle inflation, distance-field wall penalties, and path simplification. The final planner operates on the ROS costmap, receives start and goal poses from `move_base`, performs A* search over an 8-connected grid, rejects non-traversable cells, prevents diagonal corner-cutting, applies costmap penalties, simplifies the path using line-of-sight checks, and publishes the resulting global plan on `/ris_global_plan`.

Marker-based goal execution was implemented using `marker_goal_navigator.py`. This node subscribes to `/target_marker_id`, loads the corresponding marker position from `aruco_waypoints.yaml`, constructs a `MoveBaseGoal`, and sends it to the `move_base` action

server. This allowed the robot to navigate using semantic commands such as marker IDs instead of manually entered coordinates.

The full integrated workflow was therefore:

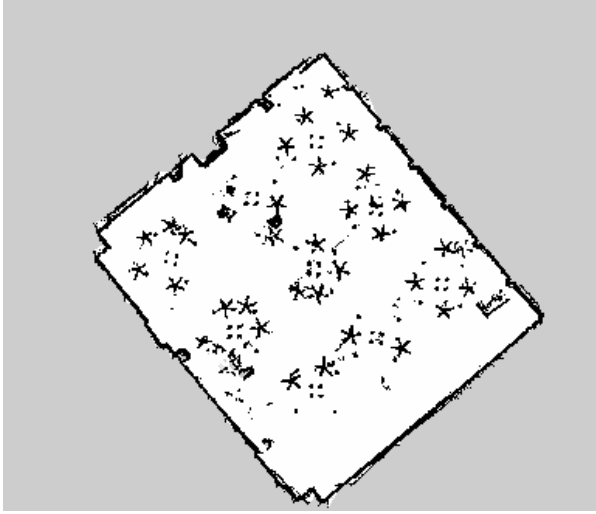
1. run `slam_and_vision.launch` to build the occupancy grid map and save ArUco marker waypoints;
2. save the map as `mymap.pgm` and `mymap.yaml`;
3. save marker positions in `aruco_waypoints.yaml`;
4. run the navigation launch file to load the saved map and marker database;
5. initialize localization using AMCL and ArUco bootstrap localization;
6. publish a target marker ID on `/target_marker_id`;
7. convert the marker ID into a map-frame waypoint;
8. send the waypoint to `move_base`;
9. generate a global path using the custom A* planner;
10. execute the path using the local planner and velocity commands.

This two-phase design connected SLAM, vision, localization, path planning, and navigation into one reusable workflow. The first phase created the map and semantic waypoint database, while the second phase used those outputs to navigate from one mapped marker location to another, such as from marker 1 to marker 2.

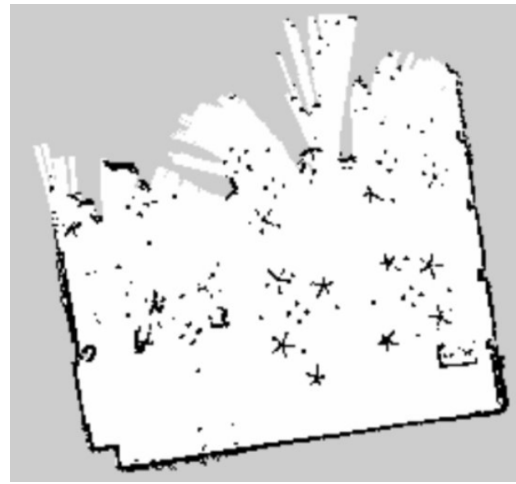
4 Results

This where the results are

4.1 SLAM and Autonomous Exploration



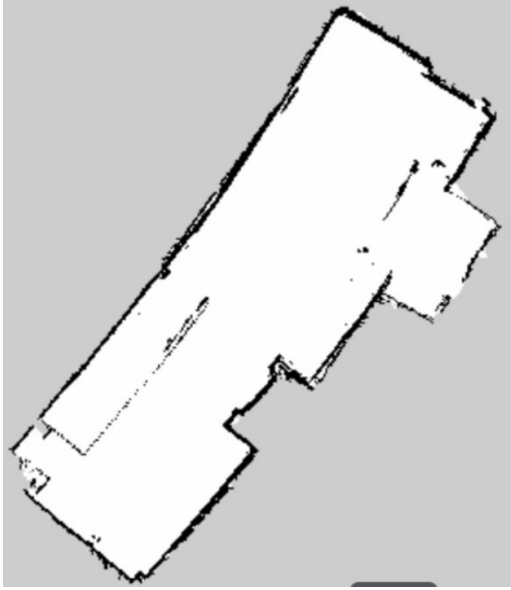
(4.1a) Using `youssef_slam` node.



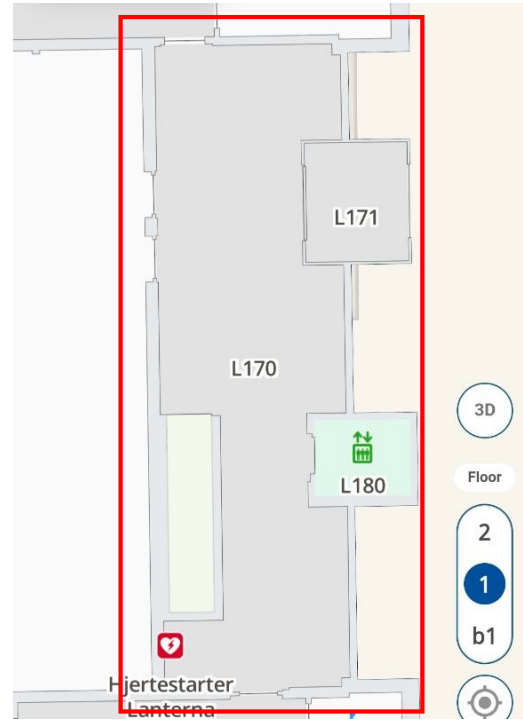
(4.1b) Using `youssef_slam` & `auto_explorer` nodes.

Figure 4.1: Evaluation of mapping performance under manual control and the `auto_explorer` node within a cluttered workspace containing dense obstacles.

Figure 4.14.1a displays the map generated through manual control, where the robot was steered by a human operator while the `youssef_slam` node monitored the process. In contrast, Figure 4.14.1b demonstrates the results obtained using the `auto_explorer` node, which navigated the cluttered environment autonomously to complete the mapping task without human intervention of same room.



(4.2a) Mapping using `auto_explorer` node.



(4.2b) Lanterna building (NTNU Ålesund) obtained from MazeMap.

Figure 4.2: Comparison of the SLAM-generated occupancy grid (a) against the official MazeMap architectural layout (b) for a sparse, low-clutter environment.

Figure 4.24.2a shows the occupancy grid generated using the `auto_explorer` node in a sparse, low-clutter environment, where the map exhibits clear wall structures and consistent spatial layout. Figure 4.24.2b presents the corresponding architectural layout of the Lanterna building obtained from MazeMap, with the explored region highlighted. A visual comparison between the two figures indicates that the generated map captures the overall geometry of the environment, including corridor shapes and relative proportions, with only minor irregularities along some boundaries.

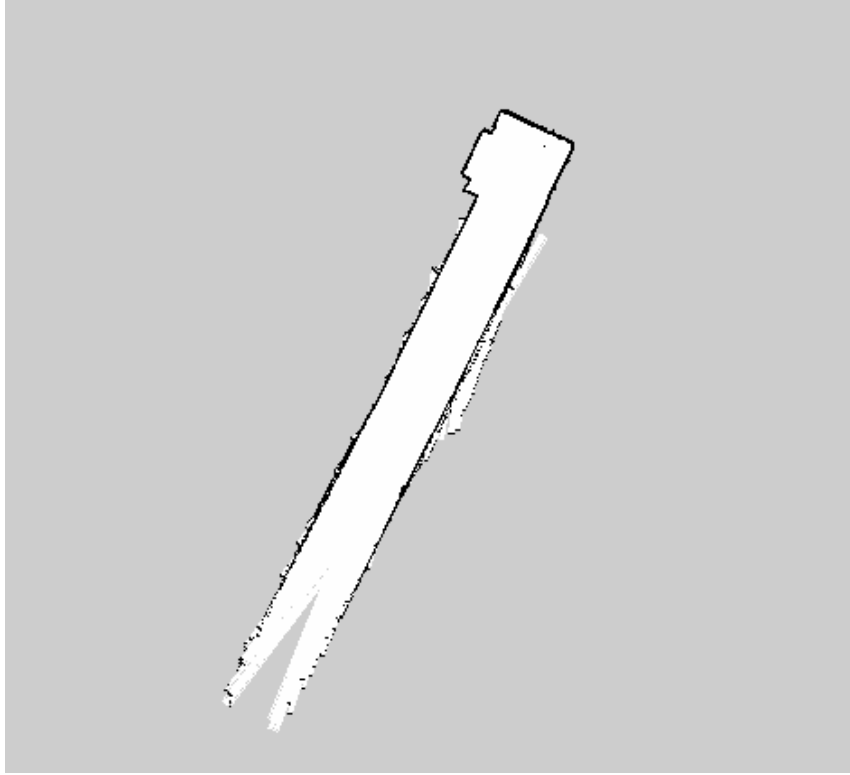


Figure 4.3: Corridor mapping using the autonomous exploration (`auto_explorer`) node.

Figure 4.3 illustrates the occupancy grid generated during autonomous exploration of a corridor using the `auto_explorer` node. The map captures the elongated corridor structure with clearly defined parallel walls and a consistent overall geometry. The explored trajectory is visible along the corridor length; however, the mapping process terminates before the full extent of the corridor is covered, leaving the far end partially unmapped. Minor irregularities and sparse noise are also present along some edges.

4.2 CV

4.3 Path planning results

The performance of the path planning algorithm was evaluated through a series of experiments in the SLAM-generated environment. The tests were conducted using different configurations of the planner, progressively adding improvements to the basic A* algorithm.

Visualization note For clarity in the RViz visualizations, the displayed grid does not use the original map resolution. The actual resolution of the occupancy grid is 0.05 m/cell, but a larger cell size was used for visualization purposes to make the paths easier to see. The grid remains correctly aligned with the map, but the cell size shown in the figures is not to scale.

4.3.1 Dijkstra planner

The Dijkstra-based planner successfully generated valid paths between predefined waypoints. The resulting paths followed the manually constructed graph and connected the selected nodes without issues.

The sequence of nodes forming the path was printed in the terminal, confirming that the planner correctly computed the shortest path between the start and goal nodes.

Figure 4.4 shows example of path obtained with start node 0 and stop node 4.

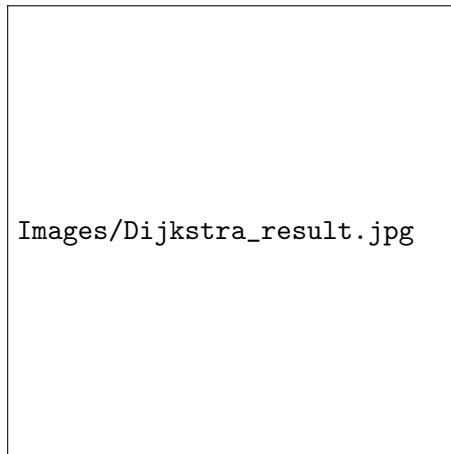
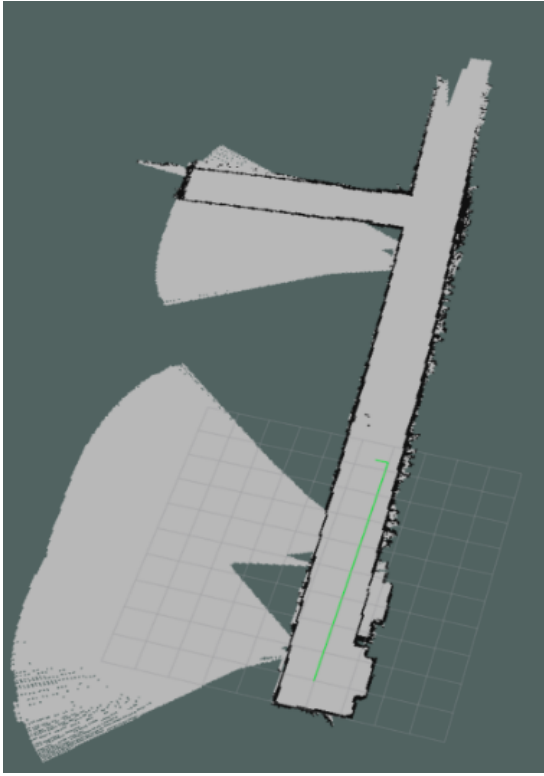


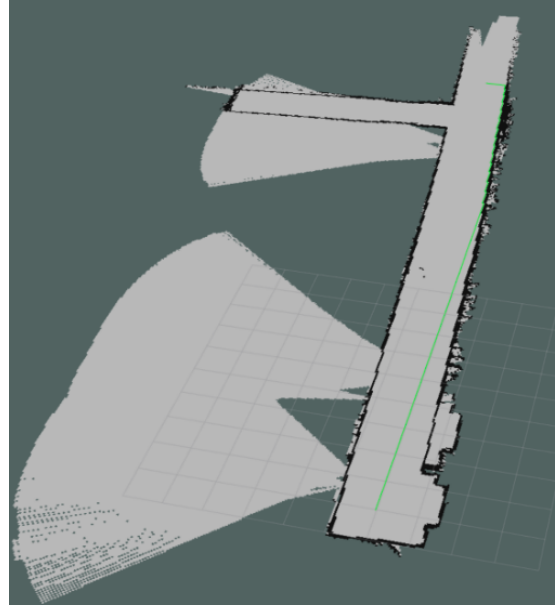
Figure 4.4: Terminal output of the Dijkstra planner showing the computed path from node 0 to node 4.

4.3.2 A* with 4-connectivity

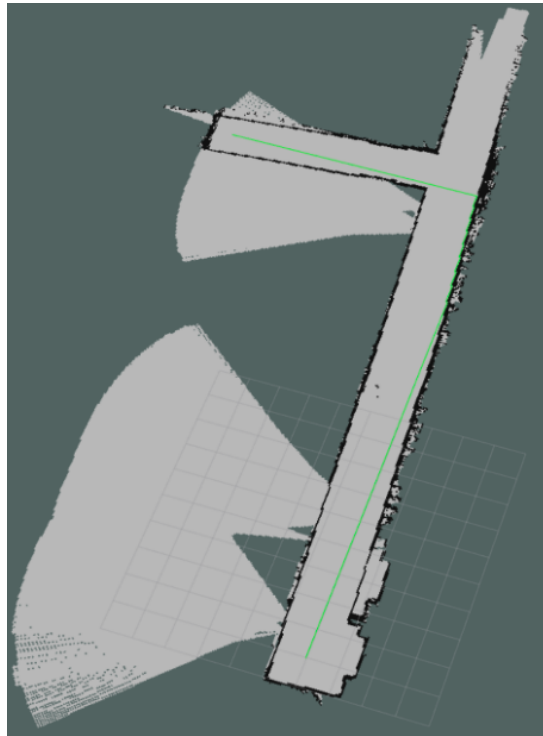
The grid-based A* planner using 4-connectivity was tested for multiple start and goal positions. The planner was able to generate valid paths in all tested cases. Figure 4.5 shows examples of paths obtained using 4-connectivity.



(4.5a) Path from node 0 to node 1.



(4.5b) Path from node 0 to node 3.

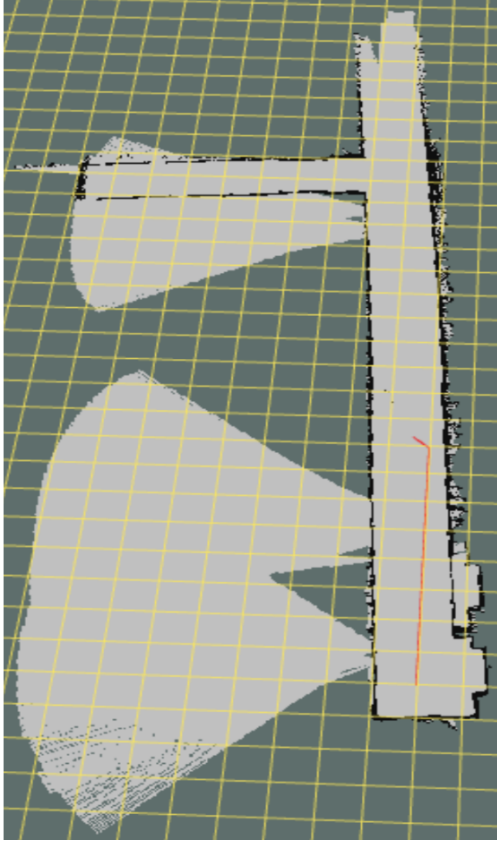


(4.5c) Path from node 0 to node 4.

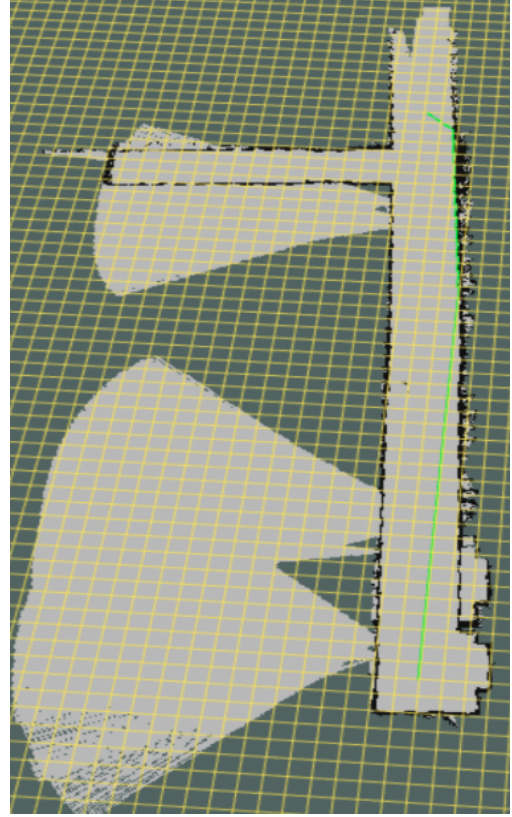
Figure 4.5: Paths generated using the 4-connectivity A* planner for three different start and goal configurations in RViz.

4.3.3 A* with 8-connectivity

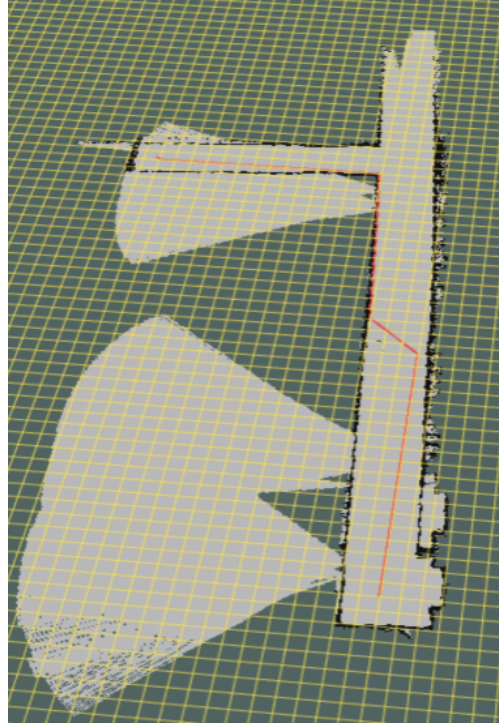
The planner was extended to 8-connectivity and tested using the same start and goal configurations as in the previous case. Figure 4.6 shows the resulting paths for these test cases.



(4.6a) Path from node 0 to node 1.



(4.6b) Path from node 0 to node 3.



(4.6c) Path from node 0 to node 4.

Figure 4.6: Paths generated using the 8-connectivity A* planner for three different start and goal configurations in RViz (same as 4-connectivity).

4.3.4 A* with local wall penalty

After adding the local wall penalty, the planner was evaluated again in the same environment. Figure 4.7 shows an example of a generated path when an additional cost is applied to cells near obstacles.

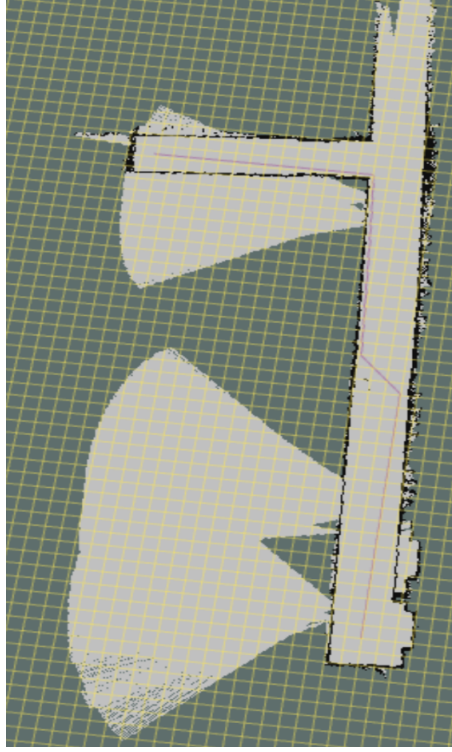
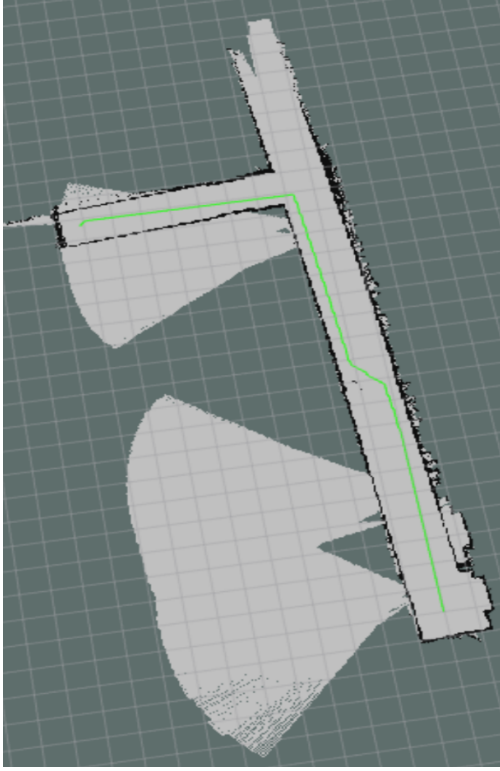


Figure 4.7: Path generated using the A* planner from node 0 to node 4 with local wall penalty in RViz.

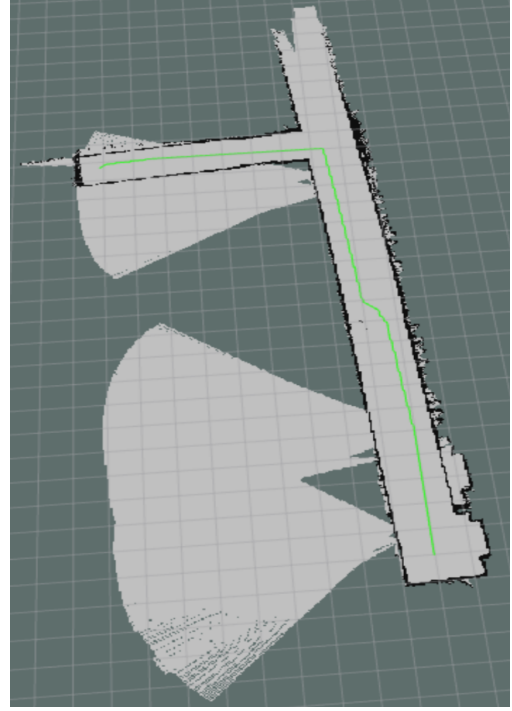
4.3.5 A* with obstacle inflation

The obstacle inflation method was tested using different inflation radii.

Figure 4.8 shows the resulting paths for two different values of the inflation radius (7 and 10 cells), illustrating the effect of increasing the safety margin around obstacles.



(4.8a) Inflation radius = 7 cells.



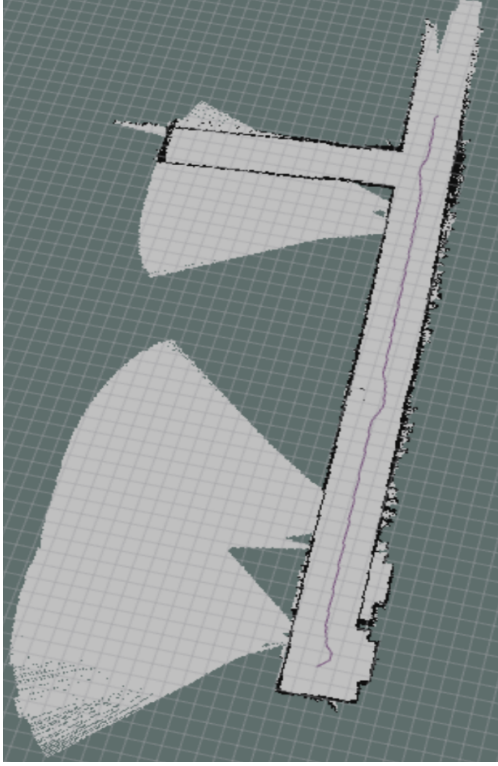
(4.8b) Inflation radius = 10 cells.

Figure 4.8: Paths generated using the A^* planner from node 0 to node 4 with obstacle inflation for different inflation radii.

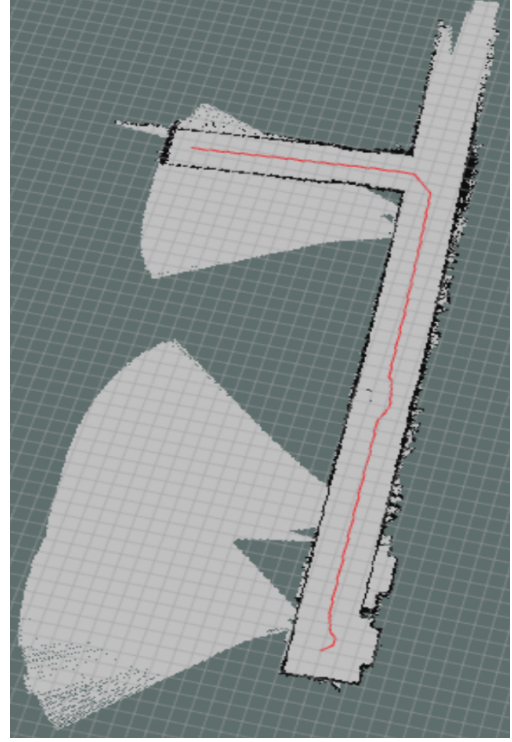
4.3.6 A^* with distance field

The final version of the planner, based on the distance field approach, was tested on the same environment.

Figure 4.9 shows the paths generated using this approach. The sequence of nodes forming the path was also printed in the terminal.



(4.9a) Path from node 0 to node 3.



(4.9b) Path from node 0 to node 4.

Figure 4.9: Paths generated using the A* planner with the distance field approach for different start and goal configurations in RViz.

The total number of nodes forming the generated paths was recorded for both the previous implementation and the final planner.

Figure 4.10 shows the corresponding terminal outputs used for this comparison, for path from node 0 to node 4.

```
jetson@ubuntu_khadija:~/Desktop/Alis4103-Jetracer-ROS-AI-KIT$ roslaunch rls grid_planner
[INFO] [1776676138.771099069]: GridPlannerNode started in testing file :) waiting for /map ...
[INFO] [1776676184.849365420]: --> origin orientation q: [x=0.000000, y=0.000000, z=0.000000, w=1.000000]
[INFO] [1776676184.849565161]: Map received =)
[INFO] [1776676184.849616816]: --> resolution: 0.050 m/cell
[INFO] [1776676184.849652138]: --> origin: (-50.000, -50.000) m
[INFO] [1776676184.849679737]: --> size: 1984 x 1984 cells
[INFO] [1776676184.849706367]: --> You gave Start cell: (919,980) and Goal cell: (1293,1162)
[INFO] [1776676185.707575836]: Published path to /planned_path
[INFO] [1776676185.707662537]: A_Star_Algo returned a path of lenght (in cells): 39
[INFO] [1776676185.707710865]: 0: (-4.025, -0.975)
[INFO] [1776676185.707750194]: 1: (3.725, -0.975)
[INFO] [1776676185.707802110]: 2: (3.775, -0.925)
[INFO] [1776676185.707825376]: 3: (-4.625, -0.925)
```

(4.10a) Total number of nodes for the previous implementation.

```
jetson@ubuntu_khadija:~/Desktop/Alis4103-Jetracer-ROS-AI-KIT$ roslaunch rls grid_planner
[INFO] [1776676500.088813230]: GridPlannerNode started in testing file :) waiting for /map ...
[INFO] [1776676500.867439402]: Distance field computed.
[INFO] [1776676500.867570418]: --> origin orientation q: [x=0.000000, y=0.000000, z=0.000000, w=1.000000]
[INFO] [1776676500.867620671]: Map received =)
[INFO] [1776676500.867682074]: --> resolution: 0.050 m/cell
[INFO] [1776676500.867741149]: --> origin: (-50.000, -50.000) m
[INFO] [1776676500.867798575]: --> size: 1984 x 1984 cells
[INFO] [1776676500.867833072]: --> You gave Start cell: (919,980) and Goal cell: (1293,1162)
[INFO] [1776676500.938075709]: Published path to /planned_path
[INFO] [1776676500.938179322]: A_Star_Algo returned a path of lenght (in cells): 99
[INFO] [1776676500.938224123]: 0: (-4.025, -0.975)
[INFO] [1776676500.938286417]: 1: (-3.725, -1.275)
[INFO] [1776676500.938320470]: 2: (-3.475, -1.275)
[INFO] [1776676500.938352037]: 3: (-3.275, -1.075)
```

(4.10b) Total number of nodes for the distance field-based implementation.

Figure 4.10: Comparison of the total number of nodes in the generated paths from node 0 to node 4 for the previous planner and the final distance field-based planner.

Table 4.1 shows how the total number of nodes differs between the two implementations.

Test	Previous planner	Distance field planner
Test 1	39	99
Test 2	XX	94

Table 4.1: Comparison of the number of nodes for different test configurations.

4.4 Navigation and Integration



Figure 4.11: Enter Caption

5 Discussion

This where the Discussion are

5.1 SLAM

5.2 CV

5.3 Path Planning

5.3.1 Dijkstra planner

The Dijkstra-based planner worked well to generate valid paths between predefined waypoints. In this implementation, the graph was manually created using only five nodes placed along the corridor. As a result, the path could only follow these predefined points.

This approach made it easy to control the path, but it also introduced some limitations. Since there were only a few nodes, the planner had very limited flexibility, and the path could not adapt to the full structure of the environment. The quality of the path depends strongly on how the waypoints are placed, and with a small number of nodes, it is difficult to represent the environment accurately.

In addition, defining the graph requires manual work, which becomes impractical for larger or more complex maps. The planner also does not consider the free space between

waypoints, since it is restricted to the predefined connections.

Overall, this approach was useful as a first step to understand graph-based planning, but it is not suitable for more general path planning tasks where flexibility and adaptability are required.

5.3.2 A* with 4-connectivity

Switching to a grid-based A* planner made the path planning more flexible, since the algorithm could now use the full occupancy grid instead of a manually defined graph. This allowed paths to be generated for any start and goal positions on the map.

However, the results show several limitations. As shown in Figure 4.5, the generated paths are valid, but they tend to follow the walls very closely. This happens because all free cells have the same cost, so the planner only minimizes the path length without considering the distance to obstacles.

Another noticeable effect is the staircase pattern in the path. Since movement is limited to horizontal and vertical directions, the planner cannot move diagonally. As a result, even when the optimal path should be a straight line, the planner approximates it using multiple straight segments. This can be seen in Figure 4.5a, where the path is composed of two straight segments instead of a single straight line.

During the analysis of the results, another observation was made regarding the alignment of the path with the environment. In RViz, the generated path appears straight in the occupancy grid, but it does not align with the actual corridor.

At first, this raised the question of whether there was a rotation mismatch between the map and the grid. To investigate this, the orientation of the map origin was checked by printing the quaternion values. The result was ($x = 0$, $y = 0$, $z = 0$, $w = 1$), which corresponds to no rotation.

This confirmed that the misalignment was not caused by an error in the transformation or in the implementation. Instead, it comes from the fact that the corridor in the real environment is not aligned with the grid axes of the occupancy map. As a result, even if the planner works correctly, the path follows the grid directions rather than the actual orientation of the corridor.

This observation helped to better understand the limitations of grid-based planning and also motivated further improvements, such as allowing diagonal movements and modifying the cost model.

Overall, this version provides a good baseline, but the paths lack smoothness and proper obstacle clearance.

5.3.3 A* with 8-connectivity

The planner was then extended to 8-connectivity, allowing diagonal movements between grid cells. The same test configurations as in the 4-connectivity case were used in order to compare the results directly.

In practice, the impact of this change was limited. As shown in Figure 4.6, two of the

tested paths are almost identical to those obtained with 4-connectivity. In one case (Figure 4.64.6a), a difference can be observed due to the possibility of diagonal movement, but the overall behavior remains similar.

The main issues observed with 4-connectivity are still present. The path continues to stay very close to the walls, since the planner still does not take obstacle proximity into account. In addition, although diagonal movement is now possible, the path still shows a staircase-like structure in several parts.

This shows that allowing diagonal movement alone is not sufficient to significantly improve the path quality. The cost model still plays a major role, and without considering the distance to obstacles, the planner continues to favor shortest paths that are not necessarily the safest or most natural.

Overall, this extension slightly increases the flexibility of the planner, but does not solve the main limitations of the previous version.

5.3.4 A* with local wall penalty

To address the issue of the path staying too close to obstacles, a local wall penalty was introduced. The idea was to increase the cost of cells that are near walls, so that the planner would prefer paths that stay further away from obstacles.

As shown in Figure 4.7, this modification leads to a small noticeable change in the path. Compared to the previous versions, the path tends to move slightly away from the walls, indicating that the additional cost is taken into account during the search.

However, the improvement remains limited. The path is still not centered in the corridor, and in some areas it continues to stay relatively close to obstacles. This is because the penalty is computed locally, based only on the immediate neighborhood of each cell.

Overall, the path is slightly improved, but still not centered.

5.3.5 A* with obstacle inflation

To further improve the distance from obstacles, obstacle inflation was added. The idea was to treat cells close to walls as occupied, so that the planner avoids them completely.

As shown in Figure 4.8, this leads to a clear increase in the distance between the path and the walls. Compared to the previous versions, the path is safer and avoids narrow areas near obstacles.

The effect of the inflation radius can also be observed. With a larger radius (10 cells), the path stays further away from the walls, while with a smaller radius (7 cells), it can still get closer to obstacles.

However, even with inflation, the path is not always centered in the corridor. The planner still mainly focuses on finding a valid path with enough clearance, rather than optimizing its position inside the free space.

Overall, this improves safety and obstacle clearance, but it does not fully solve the problem of path centering.

5.3.6 A* with distance field

In the final version of the planner, a distance field was introduced to better take into account the distance to obstacles. Instead of relying on local checks or fixed thresholds, each cell is assigned a value representing its distance to the nearest obstacle.

As shown in Figure 4.9, this leads to a clear improvement in the behavior of the planner. The generated paths are more centered in the corridor and maintain a more consistent distance from the walls. This is because the planner now considers obstacle proximity at every step, rather than only in specific areas.

Compared to the previous methods, this approach provides a more global view of the environment. The cost of each cell depends directly on its distance to obstacles, which allows the planner to naturally prefer paths that stay in the middle of free space.

Another important observation is that the path is not always perfectly straight, even in straight corridors. This can be explained by looking more closely at the environment. In Figure 5.1, the path has a slightly irregular shape at the beginning. This is due to the fact that the corridor near the start position is wider and then becomes narrower. As a result, the center of the free space is not aligned with a straight line, and the path adapts to this variation.

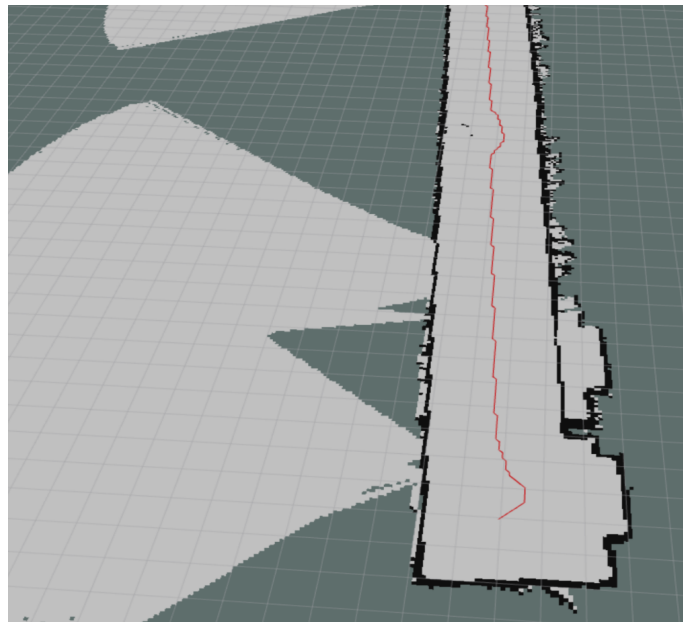


Figure 5.1: Zoomed view of the path near the start position showing the effect of corridor width variation.

A similar behavior can be observed in Figure 5.2. At the corridor intersection, the path slightly shifts to the left before continuing straight. This happens because the center of the corridor is not constant in that area. Once the corridor becomes straight again, the path recenters and follows a more regular trajectory

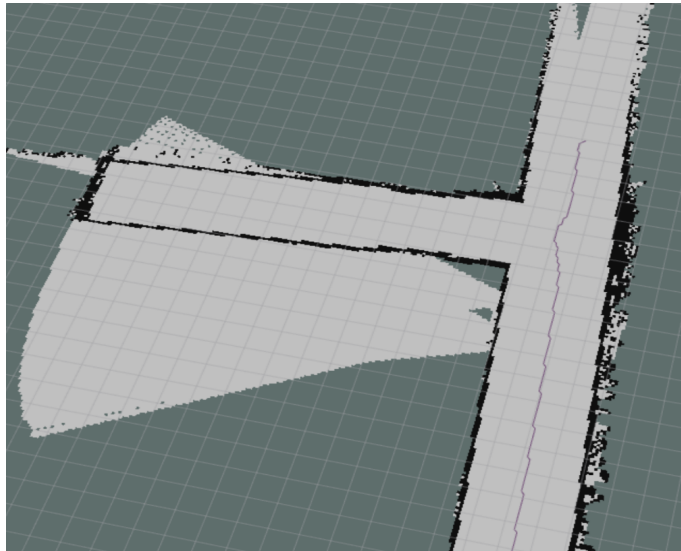


Figure 5.2: Zoomed view of the path at a corridor intersection showing the shift in the center of free space.

These observations show that the planner does not simply generate straight lines, but instead follows the center of the free space, which can change depending on the geometry of the environment.

Another important difference is the computational cost of the planner. In the previous implementation, the cost associated with each cell was computed using local neighborhood scans. This means that for each node expansion in A*, multiple surrounding cells had to be checked, which increases the computational effort.

In the final version, this computation is replaced by a distance field that is precomputed once before the search. Although this adds an initial preprocessing step, the cost of evaluating each cell during A* becomes much lower, since the distance can be directly retrieved.

As a result, the overall planning process becomes more efficient, especially when exploring a large number of nodes. At the same time, the path quality is improved, with paths that are safer and better positioned within the environment.

5.4 Navigation and Integration

6 Conclusion

This where the Conclusion is

Bibliography

- [1] Andrii Kudriashov et al. *SLAM Techniques: Application for Mobile Robot in Rough Terrain*. Vol. 87. Mechanisms and Machine Science. Springer, 2020. DOI: 10.1007/978-3-030-48981-6.
- [2] Steven M. LaValle. *Planning Algorithms*. 1st Edition. Cambridge, UK: Cambridge University Press, 2006. ISBN: 9780521862059. URL: <https://lavalle.pl/planning/>.
- [3] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Updated First Edition. Preprint version, 3rd printing 2019. Cambridge, UK: Cambridge University Press, 2017. ISBN: 9781107156302. URL: <http://modernrobotics.org>.
- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 3rd ed. Pearson, 2010. ISBN: 978-0-13-604259-4.
- [5] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Springer, 2022.
- [6] Waveshare. *JetRacer ROS AI Kit*. https://www.waveshare.com/wiki/JetRacer_ROS_AI_Kit. Accessed: 2026-04-29.
- [7] Waveshare. *JetRacer ROS AI Kit Tutorial VIII: Start the Camera Node*. https://www.waveshare.com/wiki/JetRacer_ROS_AI_Kit_Tutorial_VIII:_Start_the_Camera_Node. Accessed: 2026-04-29.
- [8] Brian Yamauchi. ‘A Frontier-Based Approach for Autonomous Exploration’. In: *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation*. 1997, pp. 146–151.

Appendices